Smart Complaint Management System: Software Requirements, Test Plan, and Architecture & Design Specification

I. INTRODUCTION

A. Problem Statement

Managing complaints manually is difficult and time-consuming. Users often do not know the status of their complaints, and administrators struggle to organise, assign and track them. SCMS provides one centralised platform where users submit and track complaints and administrators manage, assign, update and resolve them efficiently.

B. Purpose, Scope and Audience

This document specifies what SCMS must do (Section II), how it will be verified (Section III) and how it is structured (Section IV). SCMS 1.0 covers account management, complaint lifecycle management, feedback and administrative handling. Notifications by e-mail/SMS and native mobile apps are out of scope. The audience is the development team, course evaluators and testers.

C. Definitions and Acronyms

- **FR/NFR:** Functional / Non-Functional Requirement
- **SR:** Security Requirement
- **RBAC:** Role-Based Access Control
- **JWT:** JSON Web Token
- **DAL:** Data Access Layer
- **RTM:** Requirements Traceability Matrix
- **UAT:** User Acceptance Testing

D. Feasibility Study

- **Technical:** Buildable with standard web technologies and MySQL.
- **Economic:** Relies on free, open-source tools, so cost is low.
- **Operational:** The interface is simple for both users and administrators.
- **Schedule:** A four-member team can build and test it within the project timeline.

Conclusion: The system is technically, economically, operationally, and schedule feasible.

II. SOFTWARE REQUIREMENTS SPECIFICATION

A. Overall Description

1. **Product perspective:** SCMS is a new, self-contained web application: a browser client talks over HTTPS to a REST API server, which persists data in a MySQL database.
2. **User classes:**
    - **A1 (User):** Registers, submits and tracks complaints and gives feedback.
    - **A2 (Administrator):** Views, categorises, assigns, updates and resolves complaints.
3. **Operating environment:** Latest two versions of Chrome, Firefox and Edge on desktop and mobile; server on Ubuntu 22.04 with Node.js 20 and MySQL 8.
4. **Constraints and assumptions:** Only open-source tools are used; departments are pre-configured by the administrator; users have a valid e-mail address and Internet access.

Table I: Actors and Use Cases

ID	Actor	Use Cases
A1	User	Register, Login, Submit Complaint, Track Complaint, Provide Feedback
A2	Administrator	Admin Login, View Complaints, Categorize Complaint, Assign Complaint, Update Complaint, Resolve Complaint

B. Use Case Model

The system boundary contains two actors and eleven use cases. Each use case maps to at least one FR in Table II and is traced to tests in Table VIII.

C. Functional Requirements

Table II: Functional Requirements

ID	Requirement (The system shall...)
FR01	Let a new user register with name, unique e-mail and a password of > 8 characters containing a letter and a digit, and let users and administrators log in with e-mail and password; duplicate e-mails are rejected.

FR02	Let an authenticated user submit a complaint with title (< 100 chars), description (≤ 1000 chars), category and an optional JPG/PNG/PDF attachment ($\leq 5MB$).
FR03	Assign every complaint a unique ID of the form CMP-YYYY-NNNNNN at submission and display it to the user.
FR04	Show the owner of a complaint its current status (Submitted, Assigned, In Progress, Resolved, Closed) and its timestamped status history.
FR05	Let an administrator list all complaints, filter them by status, category and date range, and set or change a complaint's category.
FR06	Let an administrator assign a complaint to exactly one department from a predefined list, changing its status to Assigned.
FR07	Let an administrator change a complaint's status only along the defined workflow, with a remark (< 500 chars), and mark it Resolved with a resolution note; every change records time and admin ID.
FR08	Let the owner submit exactly one feedback (rating 1–5, optional comment < 300 chars) per complaint, only after it is Resolved.

D. Non-Functional Requirements

Table III: Non-Functional Requirements

ID	Attribute	Measurable Requirement
NFR01	Security	Every non-public page and API shall require a valid session; access shall follow SR1–SR6.

NFR02	Usability	A first-time user shall submit a complaint in $\leq 3min$ without training; every main task shall be reachable in $\leq 3clicks$ from the dashboard; layouts shall work from 360 to 1920 px width.
NFR03	Performance	95% of login, submit and track requests shall complete in $< 2s$, with 100 concurrent users.
NFR04	Reliability	No committed complaint data shall be lost: all writes are transactional, availability $\ge99\%$ per month, daily backups kept for 7 days.
NFR05	Privacy	A complaint and its owner's personal data shall be visible only to that owner and to administrators; data shall travel only over HTTPS.
NFR06	Maintainability	Code shall be layered so one module changes without editing others; service-layer unit-test coverage $>70\%$; zero ESLint errors.

E. External Interface Requirements

- **User Interface:** Responsive web pages with separate User and Administrator dashboards.
- **Software Interfaces:** REST/JSON API and MySQL 8 via a pooled driver.
- **Communication:** HTTPS (TLS 1.2 or higher) only.

F. Security Requirements

Security Objectives:

- **SO1 Confidentiality:** Only authorised parties can read user and complaint data.
- **SO2 Integrity:** Complaint records and status history cannot be altered by unauthorised actors.
- **SO3 Accountability:** Every administrative action is attributable to a specific administrator.

Security Requirements:

- **SR1:** Passwords shall be stored only as bcrypt hashes (cost > 10) and never logged.

- **SR2:** Sessions shall use signed JWTs that expire 60 minutes after issue.
- **SR3:** RBAC and ownership checks shall be enforced server-side on every API call; violations return 403/404.
- **SR4:** Five consecutive failed logins shall lock the account for 15 minutes.
- **SR5:** All inputs shall be validated server-side, all SQL shall use parameterised queries, and output shall be encoded to prevent injection and XSS.
- **SR6:** All traffic shall use HTTPS, and every admin action shall be written to an audit log (who, what, when).

(Mapping: SR1, SR2 and SR4 realise SO1; SR3 and SR5 realise SO1 and SO2; SR6 realises SO3.)

III. TEST PLAN

1. **Introduction:** Test plan ID SCMS-TP-1.0. Verifies FR01–FR08, NFR01–NFR06, and SR1–SR6 against the design.
2. **Test Items:** SCMS 1.0 build including React web client, Node.js/Express API (Auth, Complaint, Assignment, Feedback services), middleware, DAL, and MySQL schema.
3. **Features to be Tested:** Registration and login (FR01); complaint submission and ID generation (FR02, FR03); tracking (FR04); admin view, filter and categorise (FR05); assignment (FR06); status workflow and resolution (FR07); feedback rules (FR08); access control, privacy and security controls (NFR01, NFR05, SR1–SR6); response time (NFR03); data durability (NFR04); usability (NFR02); maintainability (NFR06).
4. **Features Not to be Tested:** E-mail/SMS notifications, native mobile apps, loads above 100 concurrent users, legacy browsers, infrastructure penetration testing.
5. **Approach:**
    - **Unit Tests (Jest):** Cover service-layer logic ($>70\%$ statement coverage).
    - **Integration Tests (Supertest):** API endpoint tests using seeded database.
    - **System Tests (Selenium):** End-to-end browser workflows.
    - **Performance Tests (JMeter):** Load and concurrent user testing.
    - **Usability Testing:** Timed tasks with 5 new users.
    - **Design Techniques:** Equivalence partitioning, boundary-value analysis (BVA), state-transition testing.

    5.1 Security Validation: Tests include brute-force lockout (SR4), expired tokens (SR2), password hash verification (SR1), vertical/horizontal escalation checks (SR3), injection/XSS payloads (SR5), HTTP-to-HTTPS redirect (SR6), and OWASP ZAP baseline scans.
6. **Pass/Fail Criteria:** Actual result equals expected result. Release requires 100% security and high-priority test pass rate, $>95\%$ overall pass rate, and zero open critical/high defects.
7. **Suspension and Resumption:** Suspended if smoke test fails or database is unavailable; resumes upon passing smoke tests on fixed build.
8. **Environment & Deliverables:** Ubuntu 22.04, Node.js 20, MySQL 8 with synthetic data (1000 complaints, 50 users). Tools: Jest, Supertest, Selenium, JMeter, OWASP ZAP. Deliverables: Test plan, test cases, test logs, defect reports, and summary report.

9. **Responsibilities and Schedule:**
    - **N. Jeevan Naga Deep:** Test lead and security tests.
    - **M. Nikhil Sai:** Unit and integration tests.
    - **Nandakishore PV:** System and usability tests.
    - **N. K. M. S. Akhil:** Performance and reliability tests.
    - *Timeline:* Week 1 Unit, Week 2 Integration, Week 3 System/Security/Performance, Week 4 UAT & Summary.
10. **Risks:** Schedule pressure mitigated by prioritizing security/high-priority features. Real personal data is never used.

Table IV: Test Cases (Functional, Non-Functional, Security)

TC ID	Req.	Type	Test Description / Input	Expected Result
TC01	FR01	Functional	Register with name "Ravi K", unused email & "Pass1234"; re-register with same email.	201 Created; role User; second attempt returns 409 "email already registered".
TC02	FR01	Functional	Log in with valid user credentials, then valid administrator credentials.	JWT issued; routes to correct dashboard.
TC03	FR02	Functional (BVA)	Submit complaints with 100 & 101 char titles, and 5.0 MB & 5.1 MB attachments.	100 chars & 5.0 MB accepted; 101 chars & 5.1 MB rejected with 422.
TC04	FR02, FR03	Functional	Submit two valid complaints sequentially.	Both saved (201); distinct CMP-YYYY-

				NNNNNN IDs displayed.
TC05	FR04	Functional	Owner opens complaint moved to Assigned and In Progress.	Status "In Progress" displayed with 3 timestamped history entries.
TC06	FR05, FR06, SR6	Functional	Admin filters status=Submitted, sets category "Electrical", assigns to "Maintenance".	Submitted listed; status updated to Assigned; history and audit-log rows updated.
TC07	FR07	Functional (State)	Admin moves Assigned → In Progress → Resolved with remarks; attempts Resolved → Submitted.	Valid transitions saved; invalid transition returns 409 and status remains unchanged.
TC08	FR08	Functional (BVA)	Owner rates Resolved complaint 4, re-rates it, rates unresolved complaint, submits rating 6.	First feedback 201; repeat 409; unresolved 409; rating 6 returns 422.

TC09	NFR01, SR2, SR3	Security	Call PATCH /complaints/{id}/assign with User token; call protected API with no token or 61-min expired token.	403 for User token; 401 for missing/expired token; no data modified.
TC10	NFR05, SR3	Security	User A requests GET /complaints/{id} for a complaint owned by User B.	404 returned; response contains no field of B's complaint.
TC11	SR1, SR4	Security	Enter wrong password 5 times, then correct one; inspect users table.	Account locked for 15 min (423); only bcrypt hashes stored.
TC12	SR5, SR6	Security	Login email OR 1=1; description <script>alert(1)</script>; plain HTTP request; OWASP ZAP scan.	Login rejected; script escaped; HTTP redirects to HTTPS via 301; zero High/Medium ZAP alerts.
TC13	NFR03	Performance	JMeter: 100 concurrent virtual users for 10 min (60% track, 30% submit, 10% login).	95th-percentile response time $< 2s$; error rate $<1\%$.

TC14	NFR04	Reliability	Stop API server during submission and restart; restore database from daily backup.	Partial rows rolled back; committed data intact; successful restore.
TC15	NFR02	Usability	Five first-time users submit and track a complaint without training.	All submit within 3 min; tasks reached in < 3 clicks.
TC16	NFR06	Maintainability	Run ESLint and Jest coverage; add comment-length rule to Feedback service and re-run suite.	Zero lint errors; service coverage $\ge70\%$; only Feedback files modified; tests pass.

IV. SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION

A. Architectural Pattern

SCMS uses a layered (three-tier) client-server architecture:

1. **Presentation Tier:** React single-page application (SPA).
2. **Application Tier:** Node.js/Express (Routes → Middleware → Services → DAL).
3. **Data Tier:** MySQL database.

Each layer only calls the layer directly below it to ensure separation of concerns, single enforcement point for security (NFR01, SR3), and easy maintainability (NFR06).

B. Component Description and Traceability

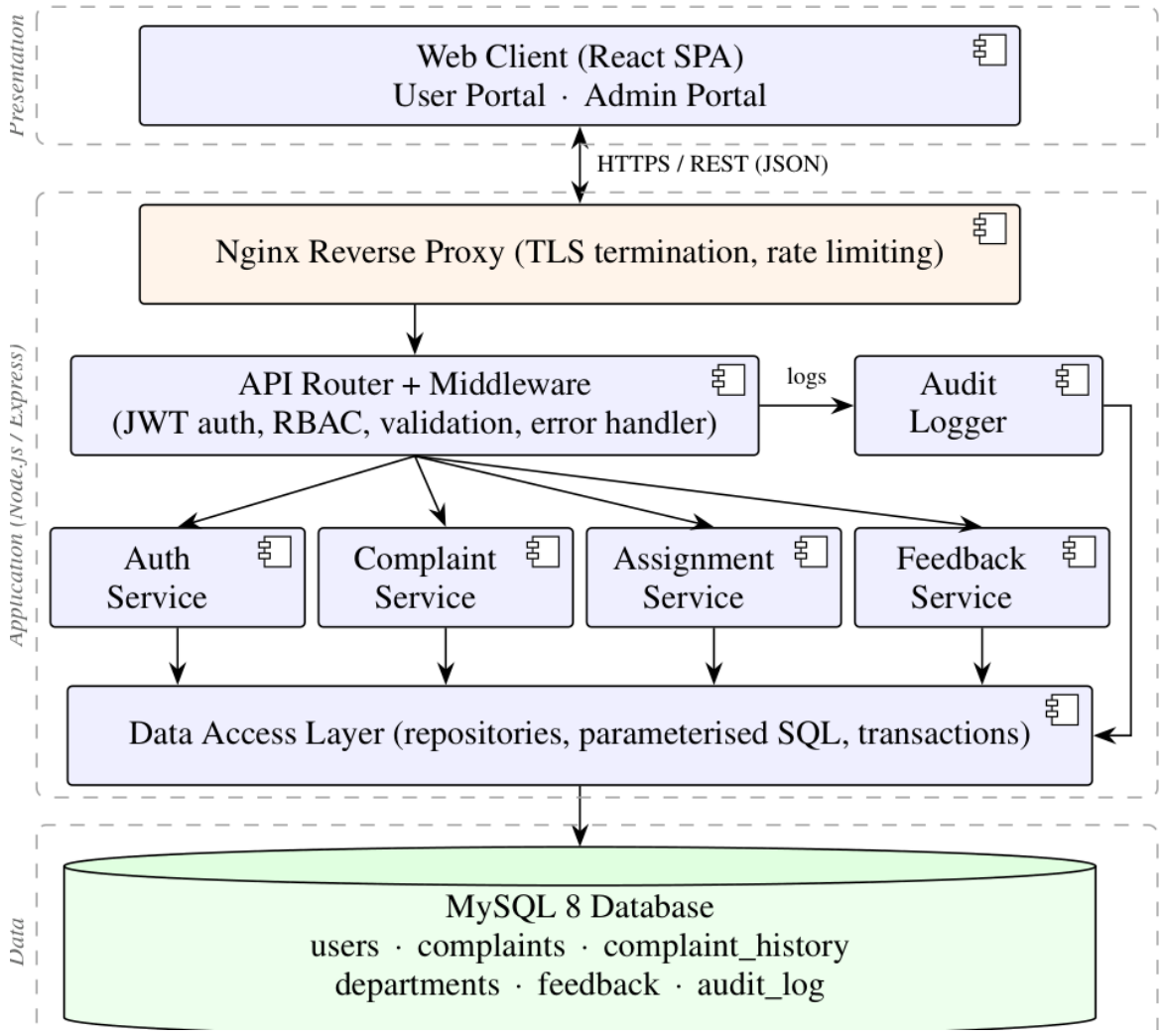


Fig. 2. UML component diagram of SCMS (three tiers).

Table V: Component Description and Requirement Traceability

Component	Responsibility	Realises
Web Client	User and Admin pages, client-side validation, output encoding	FR01–08, NFR02
Nginx Proxy	TLS termination, HTTP → HTTPS redirect, rate limiting	SR6, NFR03

API Router + Middleware	Routing, JWT verification, RBAC, input validation, central error handling	NFR01, SR2, SR3, SR5
Audit Logger	Records who did what and when for admin actions	SR6, FR07
Auth Service	Registration, login, bcrypt hashing, lockout	FR01, SR1, SR4
Complaint Service	Submission, ID generation, tracking, categorisation, status workflow, ownership checks	FR02–05, FR07, NFR05
Assignment Service	Department list and complaint assignment	FR06
Feedback Service	One feedback per resolved complaint	FR08
DAL	Repositories, parameterised queries, transactions	NFR04, SR5
MySQL DB	Persistent storage, daily backups	NFR04

C. Security Architecture

- **Transport:** Nginx terminates TLS 1.2+ and redirects HTTP to HTTPS (SR6).
- **Authentication:** Auth Service hashes passwords with bcrypt and issues HS256 JWTs (60-min expiry) stored in environment variables (SR1, SR2, SR4).
- **Authorisation:** Middleware verifies roles; Complaint Service enforces ownership (returns 404 for unauthorized access) (SR3, NFR05).
- **Input Validation:** Middleware schema validation, parameterised SQL in DAL, and React output encoding (SR5).

- **Data Security:** Least-privilege DB accounts without DDL rights (NFR04).
- **Accountability:** Audit Logger tracks administrative actions (SO3).

D. Other Architectural Views

- **Deployment:** Browser → Nginx Proxy → Node.js Application Server → MySQL Database (Application and DB hosted on private network).
- **Data Schemas:**
  - USER(user_id, name, email, password_hash, role, failed_attempts, locked_until)
  - COMPLAINT(complaint_id, user_id, title, description, category, status, dept_id, created_at)
  - COMPLAINT_HISTORY(id, complaint_id, old_status, new_status, remark, changed_by, changed_at)
  - DEPARTMENT(dept_id, name)
  - FEEDBACK(complaint_id [unique], rating, comment)
  - AUDIT_LOG(id, admin_id, action, target, at)
- **Complaint States:** Submitted → Assigned → In Progress → Resolved → Closed (after feedback or 7 days).

E. API Design

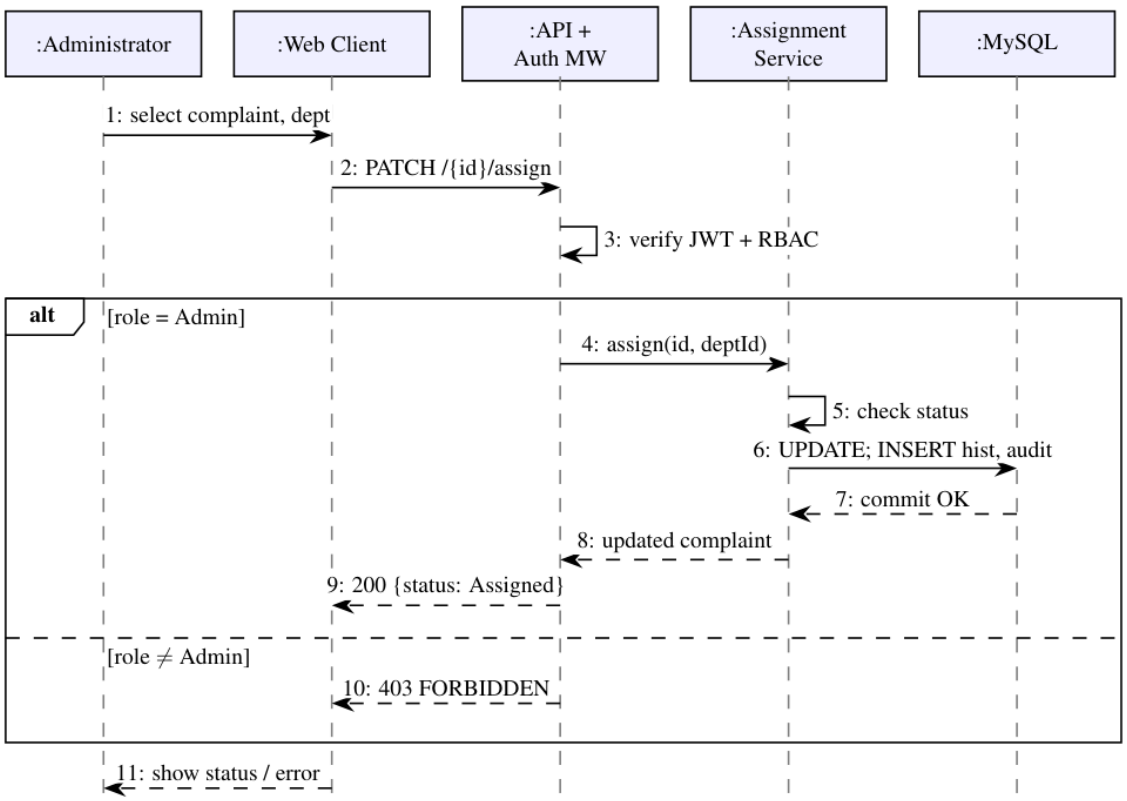


Fig. 4. Sequence diagram: Assign Complaint with RBAC check.

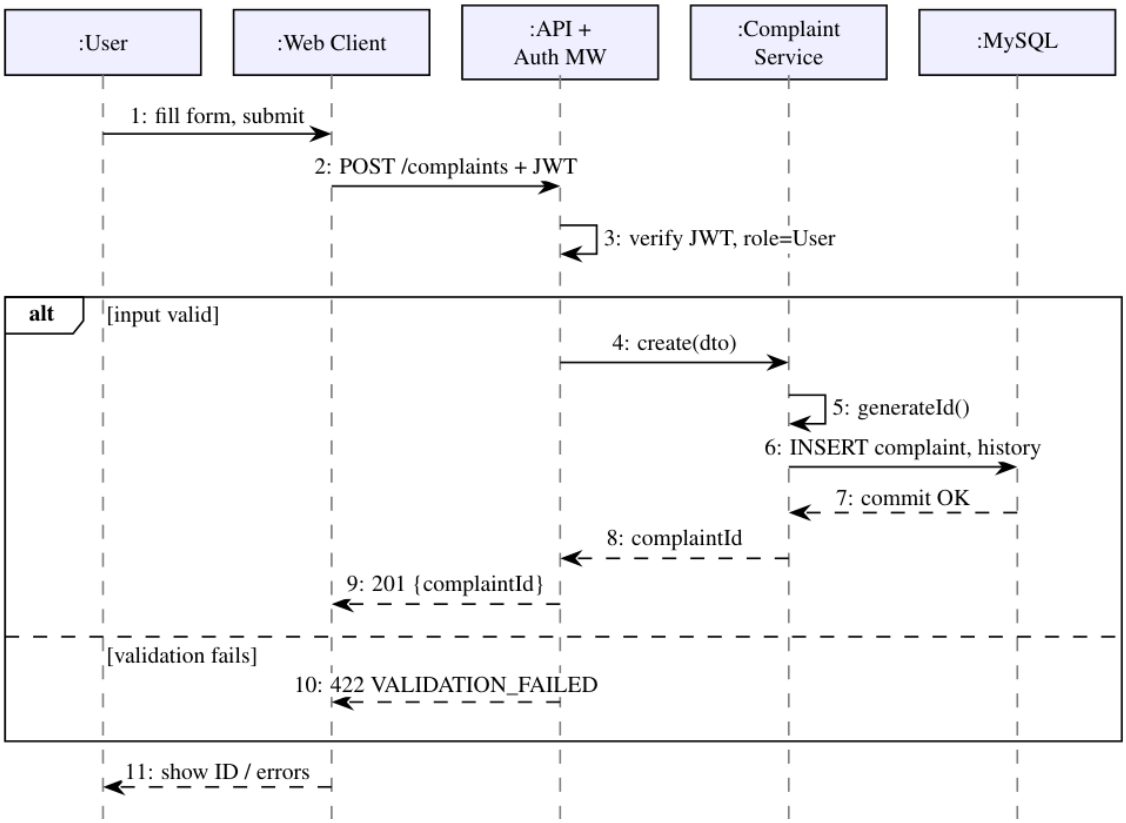


Fig. 3. Sequence diagram: Submit Complaint.

Table VI: REST API Endpoints

Method	Endpoint	Role	Purpose (Success)
POST	/api/v1/auth/register	Public	Create account (201)
POST	/api/v1/auth/login	Public	Return JWT and role (200)
POST	/api/v1/complaints	User	Submit, return ID (201)

GET	/api/v1/complaints	User/Admin	Own list / all with filters (200)
GET	/api/v1/complaints/{id}	Owner/Admin	Details and history (200)
PATCH	/api/v1/complaints/{id}/category	Admin	Set category (200)
PATCH	/api/v1/complaints/{id}/assign	Admin	Assign department (200)
PATCH	/api/v1/complaints/{id}/status	Admin	Update/resolve with remark (200)
POST	/api/v1/complaints/{id}/feedback	Owner	Rating and comment (201)

F. Error Handling

Table VII: Error Code Mapping

HTTP Code	Label	When Triggered
400	BAD_REQUEST	Malformed JSON or parameters
401	UNAUTHENTICATED	Missing, invalid, or expired JWT

403	FORBIDDEN	Role not allowed for the action
404	NOT_FOUND	Resource absent or not owned by caller
409	CONFLICT	Duplicate e-mail/feedback, invalid state transition
422	VALIDATION_FAILED	Field rules violated (length, type, size)
423	ACCOUNT_LOCKED	Lockout after 5 failed logins
429	RATE_LIMITED	Too many requests from one client
500	INTERNAL_ERROR	Unexpected failure (details logged only)

V. REQUIREMENTS TRACEABILITY MATRIX

Table VIII: Requirements Traceability Matrix

Req.	Component	Use Case	API	Tests
FR01	Auth Service	Register, Login, Admin Login	/auth/*	TC01, TC02
FR02	Complaint Service	Submit Complaint	POST /complaints	TC03, TC04

FR03	Complaint Service	Submit Complaint	POST /complaints	TC04
FR04	Complaint Service	Track Complaint	GET /complaints/{id}	TC05
FR05	Complaint Service	View, Categorize	GET /complaints, PATCH /category	TC06
FR06	Assignment Service	Assign Complaint	PATCH /assign	TC06
FR07	Complaint Svc, Audit Logger	Update, Resolve	PATCH /status	TC07
FR08	Feedback Service	Provide Feedback	POST /feedback	TC08
NFR01	Middleware	All (access control)	All protected APIs	TC09
NFR02	Web Client	All	All	TC15
NFR03	All Tiers	All	All	TC13
NFR04	DAL, MySQL	Complaint Management	Write APIs	TC14

NFR05	Middleware, Complaint Service	Track, View	GET /complaints/{id}	TC10
NFR06	Layered Structure	System Maintenance	All	TC16
SR1, SR4	Auth Service	Register, Login	/auth/*	TC11
SR2, SR3	Middleware	All protected	All	TC09, TC10
SR5	Middleware, DAL	Login, Submit	All	TC12
SR6	Nginx, Audit Logger	Admin use cases	All	TC06, TC12

VI. CONCLUSION

This report delivers the Part-1 artefacts for SCMS: an IEEE-format SRS with eight measurable FRs, six NFRs, a use case model and a security section with three objectives and six requirements; an IEEE 829 test plan with a dedicated security-validation approach and sixteen test cases; and an architecture and design specification with a layered architecture, component and sequence diagrams, security architecture, API design and error-handling strategy. The RTM confirms that every requirement is realised by a component and verified by at least one test.